

Structuring big WSGI applications

The WSGI spec is quite flexible as there is only one entry point into your application. You can, therefore, write the app as a single function, a class implementing the `__call__()` method, or even as a complete module. When you get around to writing bigger WSGI applications (and this applies not only to pure WSGI applications, but also to micro frameworks that expose the application as a WSGI callable—including Flask, Pylons, WebOb, and even Tornado in WSGI mode), you'll generally want to write them as a module.

There are a couple of reasons for this. First, it's always a good idea to write self-contained code. A well written WSGI application can be distributed using *distribute*, along with a *requirements.txt* file specifying dependencies for *pip* to download. In fact, because it's so easy to get WSGI apps on the Internet for free (using Google App Engine or Heroku), and because these services generally mandate writing system independent modular code, it is a good idea to write code that can be run out-of-the-box on these services. Second, wouldn't it be awesome to do the following:

```
from MySuperAwesomeWSGIApp import app

# Here's its important to note exactly what app is
# app is your WSGI callable. Its definition and
# initialisation resides in __init__.py. Yes,
# initialisation. The app should be fully init-ed
# and ready to be served.

import OneMiddleware
import AnotherMiddleware
import ThirdMiddleware

app = OneMiddleware(app)
app = AnotherMiddleware(app)
app = ThirdMiddleware(app)

# Now hook up the app to the server - more on that later
```

Because WSGI apps can be called by another WSGI app (as is evident in how it is used in middleware), you can hook up your WSGI app (which you've thoughtfully written as a module) to other WSGI apps. But, to which ones? Well, there are URL-based routers, sub domain-based routers and authentication filters that are widely available online. You will find links to a few in the resource section.

Serving WSGI applications

Most people try serving WSGI applications using Apache and *mod_wsgi*. They fail miserably and complain that Python is slow or that WSGI is dead technology. What they don't realise is that they're serving it wrong.

While it's perfectly acceptable to make a WSGI app face the Web directly using *mod_wsgi*, it isn't exactly advisable for performance and security considerations. Security wise, it's a bad idea to run the Python interpreter inside your Web server's process. You will generally want to serve your apps with a pure-Python server and put a *nginx* reverse-proxy in front. With that in mind, let's examine a few WSGI servers and look at how to use them.

First is Python's own *wsgiref*. This is the reference WSGI server implementation, which comes as a part of the Python standard library. It is thread-pooled, reasonably fast, and great for testing, development, or even running as a production server on your intranet (although for a particularly busy intranet that might be pushing it a bit). Use it as follows:

```
from wsgiref.simple_server import make_server
from MyWSGIApp import app

httpd = make_server('', 5000, app)
httpd.serve_forever()
```

When you are ready to move up to the production level (i.e., on the real Internet), you will want to use a more robust server. There are a few options to choose from. You can go threadpooled, event-based, or use the current rage in Python—application level micro-threads that are co-operatively scheduled. Python calls them greenlets, and they are great for implementing asynchronous TCP servers, such as the one which comes with *gevent*. *Gevent* can be installed from the repos or from *pip*, and the server set up is as follows:

```
from gevent.wsgi import WSGIServer
from MyWSGIApp import app

httpd = WSGIServer('', 5000), app)
httpd.serve_forever()
```

Note that the host-port pair is passed as a tuple. *Gevent*'s greenlet-based server is currently regarded as the fastest Python implementation of the WSGI server, and because the server itself is written in C (it's actually written using *libev*), it really is quite fast.

A different way of serving up a WSGI application is to use Green Unicorn, which is actually ported over from Ruby's Unicorn project. It's a pre-forking threadpooled server, but it can make use of *gevent* and greenlets. To use Green Unicorn, the *gunicorn* package is installed from *pip* or your package manager (ensure you install *gevent* because that is an optional dependency). Serving the application is a little different however—Green Unicorn is a command line application, so you need to use the following command:

```
$: gunicorn -k gevent -w 4 -b 127.0.0.1:5000 MyWSGIApp:app
```